COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	8/5/2026	**DATE SUBMITTED:**	8/14/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 5.1 **Queues**

ILO.A

CODE

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main () {

  //create an object
  std::queue<char> myQ;

  //use the push operation
  myQ.push('C');
  myQ.push('E');
  myQ.push('D');
  myQ.push('R');
  myQ.push('I');
  myQ.push('C');
  std::cout<< "current FRONT is " << myQ.front()<<std::endl;
  std::cout<< "current BACK is " << myQ.back()<<std::endl;

  //use thge  dequeue operation
  myQ.pop();
  display(myQ);
  myQ.pop();
  display(myQ);
  myQ.pop();

  //check the queue if it is empty
  std::cout<<"is the queue empty?" <<std::endl;

  return 0;
}

//note only use the member function of the queue STL
void display(std::queue<char> copyQ) {

  //create copy of the queue
  std::queue<char> temp = copyQ;

  //loop until empty
  while(!temp.empty()){
    //display the front
    std::cout<<" " << temp.front() << " ";
    //dequeue the fron
    temp.pop();
  }
  //add a new line
  std::cout << std::endl;
}
```

OUTPUT

```
current FRONT is C
current BACK is C
 E  D  R  I  C
 D  R  I  C
is the queue empty?
```

COMPUTER ENGINEERING

QUEZON CITY

Observation:

The program demonstrates how queue and its following operation works from inserting and removing and how they follow the rule first in, first out was demonstrated in the code where in the first value that was removed by the pop was the first C in C, E, D, R, I, C leaving only E, D, R, I, C behind

ILO.B

CODE

MAIN.cc

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('C', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: " <<back->data<<std::endl;
    enqueue('E', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: " <<back->data<<std::endl;
    enqueue('D', &front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: " <<back->data<<std::endl;

    std::cout<<"testing the dequeue operator: \n ";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: " <<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data << " " <<"back: " <<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);

    //test Display all
    std::cout << "Queue elements: ";
    displayAll(front);

    //test if the queue is empty
    std::cout << "Is queue empty " << (isEmpty(front) ? "Yes" : "No") << std::endl;
    displayAll(front);

    return 0;
}
```

QUEUELL.h

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};

// new node creator
template <typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;

    // store the newdata to the newNode
    newNode -> data = newData;
    // point the newNode  next to null
    newNode ->next = nullptr;

    return newNode;
}

```

OUTPUT

```
front: C back: C
front: C back: E
front: C back: D
testing the dequeue operator:
 front: E back: D
front: D back: D
the queue is empty
Queue elements: Queue is empty!
Is queue empty Yes
Queue is empty!
```

```cpp
// enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr ){
    //create a new node
    qNode<T>* newNode = new_node(newData);

    //check if the queue is empty
    if((*frontPtr)==nullptr && (*backPtr)== nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
    // if not then insert at the back
    (*backPtr)->next = newNode; //LINKS THE CURRENT BACK TO THE NEW NODE
    (*backPtr)= newNode; // ASSIGNS THE BACKPTR TO THE NEW NODE
}

// dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout<< "the queue is empty"<<std::endl;
        return;
    }
    // create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to the deletTemp
    deleteTemp = (*frontPtr);

    // check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    // delete node
    delete deleteTemp;
}

// display all
template <typename T>
void displayAll(qNode<T>* front) {
    if (front == nullptr) {
        std::cout << "Queue is empty!" << std::endl;
        return;
    }

    qNode<T>* temp = front;
    std::cout << "Queue contents: ";
    while (temp != nullptr) {
        std::cout << temp->data << " ";
        temp = temp->next; // move to the next node
    }
    std::cout << std::endl;
}

// isEmpty
template <typename T>
bool isEmpty(qNode<T>* front) {
    return (front == nullptr);
}

#endif
```

COMPUTER ENGINEERING

QUEZON CITY

Observation:

The following activity showed me how to implement linked lists and things like pointers and nodes into a queue demonstrating that instead of relying on the library functions of the queue we can manage it using dynamic memory allocation.

It also made me understand that in order for the program to identify that the queue is empty we need to make it specifically point or equal to nullptr which was how I was able to do the isEmpty function by just checking the fronts value following the FIFO rule we are able to pinpoint whether the queue is empty if that value points to nullptr

For making the display all function it made me understand that unlike the standard queue made on the previous activity this time since we are using a linked list we are able to traverse through the values front to back by using next to that it doesn't delete anything on the way out

ILOA_2

CODE

MAIN.CC

```cpp
#include <iostream>
#include "queueArr.h"

int main()
{
    // creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(3);

    q.enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(5);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.enqueue(6);

    std::cout << "\nremoved: " << q.dequeue() << std::endl;
    std::cout << "removed: " << q.dequeue() << std::endl;

    // testing the clear
    q.Clear();

    std::cout << "is the queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;

    return 0;
}
```

OUTPUT

```
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is Full

removed: 1
removed: 2
is the queue empty? yes
```

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

QUEUEARR.H

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

// Array-Based Circular Queue
template <typename T>
class queueArr {
    private:
        // POINTER TO DYNAMICALLY ALLOCATE ARRAY
        T* q_arr;

        // MAXIMUM NUMBER OF ELEMENTS A QUEUE CAN HOLD
        size_t q_capacity;

        // CURRENT NUMBER OF ELEMENTS IN THE QUEUE
        size_t q_size;

        // INDEX OF THE FRONT ELEMENT IN THE QUEUE
        int q_front;

        // INDEX OF THE BACK ELEMENT IN THE QUEUE
        int q_back;

    public:

    // CONSTRUCTOR
    queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
    }

    // COPY CONSTRUCTOR
    queueArr(const queueArr& other) : q_capacity(other.q_capacity), q_size(other.q_size), q_front(other.q_front), q_back(other.q_back) {
        // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
        q_arr = new T[q_capacity];
        // COPY ELEMENTS FROM OTHER QUEUE
        for (size_t i = 0; i < q_size; ++i) {
            q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
        }
    }

    // COPY ASSIGNMENT OPERATOR
    queueArr& operator=(const queueArr& other) {
        if (this != &other) {
            // DEALLOCATE CURRENT ARRAY
            delete[] q_arr;

            // COPY CAPACITY AND SIZE
            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;

            // DYNAMICALLY ALLOCATE MEMORY FOR THE QUEUE ARRAY
            q_arr = new T[q_capacity];
            // COPY ELEMENTS FROM OTHER QUEUE
            for (size_t i = 0; i < q_size; ++i) {
                q_arr[(q_front + i) % q_capacity] = other.q_arr[(other.q_front + i) % other.q_capacity];
            }
        }
        return *this;
    }

    // DESTRUCTOR
    ~queueArr() {
        // DEALLOCATE MEMORY FOR THE QUEUE ARRAY
        delete[] q_arr;
    }

    // QUEUE OPERATIONS
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void enqueue(T value);
    T dequeue();
};

// Empty()
template <typename T>
bool queueArr<T>::Empty() {
    // CHECK IF EMPTY
    return q_size == 0;
}
```

```cpp

// Full()
template <typename T>
bool queueArr<T>::Full() {
    // CHECK IF REACHES MAX CAPACITY
    return q_size == q_capacity;
}

// Size()
template <typename T>
size_t queueArr<T>::Size() {
    // RETURN CURRENT SIZE OF QUEUE
    return q_size;
}

// Front()
template <typename T>
T queueArr<T>::Front() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the front
    return q_arr[q_front];
}
// Back()
template <typename T>
T queueArr<T>::Back() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    // Return the back
    return q_arr[q_back];
}
// Clear()
template <typename T>
void queueArr<T>::Clear() {
    // Reset the queue to its initial state
    q_size = 0;
    q_front = 0;
    q_back = -1; // Set back to -1 since there are no elements
}
// Enqueue()
template <typename T>
void queueArr<T>::enqueue(T value) {
    // Check if the queue is full
    if (Full()) {
        std::cout << "Queue is Full\n";
        return;
    }
    // Increment the back
    q_back = (q_back + 1) % q_capacity;
    // Add the new value to the back of the queue
    q_arr[q_back] = value;
    q_size++;
}
// Dequeue()
template <typename T>
T queueArr<T>::dequeue() {
    if (Empty()) {// Check if the queue is empty
        std::cout << "Queue is Empty\n";
        return T(); // Return default value of type T
    }
    T value = q_arr[q_front];// Store the front value
    // Increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    q_size--;// Decrement the size of the queue
    if (Empty()) Clear();   // Reset the indexes
    return value;// Return the dequeued value
}
#endif // QUEUEARR_H
```

COMPUTER ENGINEERING

QUEZON CITY

Observation:

This section of this activity demonstrated how we can implement a queue with the use of array more specifically the circular queue allowing old values that were popped or deleted that resulted in their values to be empty or just specifically contain a null to be reused it does the following by making each the tail of the pointer to point back to the head and since they follow the First in First out rule it enables them to be able to loop back to the front

SUPPLEMENTARY

CODE

Main.cc

```
#include <iostream>
#include "QueuePrint.h"

int main() {
    // Queue instance managing Job elements
    PrinterQueue<Job> printer;

    // Variables for storing user inputs
    std::string username;
    int pages;
    int userChoice;
    int currentID = 1; // Counter to generate incremental IDs for each job

    bool isRunning = true;
    while (isRunning) {
        // Display interactive menu
        std::cout << "1. Add new job\n";
        std::cout << "2. Exit\n\n";
        std::cout << "Enter choice: ";
        std::cin >> userChoice;

        switch (userChoice) {
```

OUTPUT

Input

```
1. Add new job
2. Exit

Enter choice: 1
Enter Username: Cedric
Enter number of pages: 12
1. Add new job
2. Exit

Enter choice: 1
Enter Username: Fernandez
Enter number of pages: 5
1. Add new job
2. Exit

Enter choice: 1
Enter Username: Kola
Enter number of pages: 2
1. Add new job
2. Exit

Enter choice: 1
Enter Username: Last
Enter number of pages: 6
1. Add new job
2. Exit

Enter choice: 2
```

```
            case 1:
                // Get job details from
user
                std::cout << "Enter
Username: ";
                std::cin >> username;
                std::cout << "Enter
number of pages: ";
                std::cin >> pages;

                // Create a new Job and
insert it into the printer queue

printer.enqueue(Job(currentID++,
username, pages));
                break;

            case 2:
                // Process and dequeue
every pending job before terminating
                while
(!printer.isEmpty()) {
                    std::cout <<
"\nQueue:\n";
                    printer.display();
                    std::cout <<
"\nProcessing the top...\n";
                    printer.dequeue();
                }
                std::cout << "Goodbye";
                isRunning = false; //
Ends the menu loop
                break;

            default:
                // Displays error if
user inputs invalid command
```

Output

```
Queue:
Job 1 | Cedric | 12 pages -> Job 2 | Fernandez | 5 pages -> Job 3 | Kola | 2
pages -> Job 4 | Last | 6 pages

Processing the top...

Queue:
Job 2 | Fernandez | 5 pages -> Job 3 | Kola | 2 pages -> Job 4 | Last | 6 pag
es

Processing the top...

Queue:
Job 3 | Kola | 2 pages -> Job 4 | Last | 6 pages

Processing the top...

Queue:
Job 4 | Last | 6 pages

Processing the top...
Goodbye
PS C:\Users\csf18\Desktop\HOA5PI>
```

```
                std::cout << "Invalid
choice. Try again.\n";
                break;
        }
    }

    return 0;
}
```

QueuePrint.h

```
#ifndef PRINTQUEUE_H
#define PRINTQUEUE_H
#include <iostream>
#include <string>

// Class representing an individual
print job
class Job {
private:
    int jobID;          // Unique
identifier for the job
    std::string user;  // Name of the
user who submitted the job
    int totalPages;     // Number of
pages to print

public:
    // Constructor: initializes job ID,
user name, and page count
    Job(int id = 0, std::string name =
"", int pages = 0)
        : jobID(id),
user(std::move(name)),
totalPages(pages) {}

    // Displays the job's details in a
format
```

```
    void printInfo() const {
        std::cout << "Job " << jobID <<
" | " << user << " | " << totalPages <<
" pages";
    }
};

// Node structure used for building the
linked list queue
template <typename T>
struct Node {
    T data;       // Holds the actual
data (e.g., a Job object)
    Node* next;  // Pointer to the next
node in the queue

    // Constructor: sets up node data
and sets next pointer to null
    Node(const T& val) : data(val),
next(nullptr) {}
};

// Queue class that manages printer
queue operations
template <typename T>
class PrinterQueue {
private:
    Node<T>* head; // Pointer to the
front of the queue (first item to
process)
    Node<T>* tail; // Pointer to the
back of the queue (new items added
here)

public:
    // Constructor: starts with an
empty queue
```

```
    PrinterQueue() : head(nullptr), tail(nullptr) {}

    // Destructor: automatically frees memory for remaining nodes when object is destroyed
    ~PrinterQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }

    // Helper method to check if the queue contains no elements
    bool isEmpty() const {
        return head == nullptr;
    }

    // Adds a new item to the end of the queue
    void enqueue(const T& newItem) {
        Node<T>* newNode = new Node<T>(newItem);

        // If queue is empty, the new node becomes both head and tail
        if (isEmpty()) {
            head = tail = newNode;
            return;
        }

        // Link old tail to new node, then shift tail pointer forward
        tail->next = newNode;
        tail = newNode;
    }
```

```
    // Removes the item at the front of
the queue
    void dequeue() {
        if (isEmpty()) {
            std::cout << "The queue is
empty" << std::endl;
            return;
        }

        // Save current head pointer to
free its memory after shifting head
        Node<T>* temp = head;
        head = head->next;

        // If queue becomes empty,
reset tail pointer as well
        if (head == nullptr) {
            tail = nullptr;
        }

        delete temp; // Free memory
    }

    // Loops through the linked list
and displays all jobs from front to
back
    void display() const {
        if (isEmpty()) {
            std::cout << "The queue is
empty" << std::endl;
            return;
        }

        Node<T>* curr = head;
        while (curr != nullptr) {
            curr->data.printInfo();
            if (curr->next != nullptr)
```

```
{
                std::cout << " -> ";
            }
            curr = curr->next;
        }
        std::cout << std::endl;
    }
};

#endif
```

Analysis

In doing this supplementary I used linked list specifically the singly linked list since each node only contains one node in addition to this I used linked list over arrays because using arrays would take a longer time to execute since the program will have to shift every remaining item over whenever the first item is removed

CONCLUSION

In this activity it helped demonstrate and teached me how a queue can be implemented using an array and how things like circular queue can make better use of available storage inside an storage through different operations like dequeue and enqueue and many more. This activity also helped me gained a deeper insight on FIFO first in, first out and how we can use that to make a circular queue that will allow the front and back position to wrap around making a loop

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance and explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation and how to do circular Linked list**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.